



This is not mandatory reading, but here's the code we'll run in the "Time Travel" videos. It may come in handy when you're doing the associated hands-on assignment.

```
SHOW TABLES;
```

```
---> set the data retention time to 90 days
```

```
ALTER TABLE TASTY_BYTES.RAW_POS.TEST_MENU SET DATA_RETENTION_TIME_IN_DAYS = 90;
```

```
SHOW TABLES;
```

```
---> set the data retention time to 1 day
```

```
ALTER TABLE TASTY_BYTES.RAW_POS.TEST_MENU SET DATA_RETENTION_TIME_IN_DAYS = 1;
```

```
---> clone the truck table
```

```
CREATE OR REPLACE TABLE tasty_bytes.raw_pos.truck_dev  
  CLONE tasty_bytes.raw_pos.truck;
```

```
SELECT
```

```
  t.truck_id,  
  t.year,  
  t.make,  
  t.model
```

```
FROM tasty_bytes.raw_pos.truck_dev t;
```

```
---> see how the age should have been calculated
```

```
SELECT
```

```
  t.truck_id,  
  t.year,  
  t.make,  
  t.model,  
  (YEAR(CURRENT_DATE()) - t.year) AS truck_age
```

```
FROM tasty_bytes.raw_pos.truck_dev t;
```

```
---> record the most recent query_id, back when the data was still correct
```

```
SET good_data_query_id = LAST_QUERY_ID();
```

```
---> view the variable's value
```

```
SELECT $good_data_query_id;
```

---> record the time, back when the data was still correct

```
SET good_data_timestamp = CURRENT_TIMESTAMP;
```

---> view the variable's value

```
SELECT $good_data_timestamp;
```

---> confirm that that worked

```
SHOW VARIABLES;
```

---> make the first mistake: calculating the truck's age incorrectly

```
SELECT
    t.truck_id,
    t.year,
    t.make,
    t.model,
    (YEAR(CURRENT_DATE()) / t.year) AS truck_age
FROM tasty_bytes.raw_pos.truck_dev t;
```

---> make the second mistake: calculate age wrong, and overwrite the year!

```
UPDATE tasty_bytes.raw_pos.truck_dev t
    SET t.year = (YEAR(CURRENT_DATE()) / t.year);
```

```
SELECT
    t.truck_id,
    t.year,
    t.make,
    t.model
FROM tasty_bytes.raw_pos.truck_dev t;
```

---> select the data as of a particular timestamp

```
SELECT * FROM tasty_bytes.raw_pos.truck_dev
AT(TIMESTAMP => $good_data_timestamp);
```

```
SELECT $good_data_timestamp;
```

---> example code, without a timestamp inserted:

```
-- SELECT * FROM tasty_bytes.raw_pos.truck_dev
-- AT(TIMESTAMP => '[insert timestamp]':TIMESTAMP_LTZ);
```

---> example code, with a timestamp inserted

```

SELECT * FROM tasty_bytes.raw_pos.truck_dev
AT(TIMESTAMP => '2024-04-04 21:34:31.833 -0700'::TIMESTAMP_LTZ);

--> calculate the right offset
SELECT TIMESTAMPDIFF(second,CURRENT_TIMESTAMP,$good_data_timestamp);

--> Example code, without an offset inserted:

-- SELECT * FROM tasty_bytes.raw_pos.truck_dev
-- AT(OFFSET => -[WRITE OFFSET SECONDS PLUS A BIT]);

--> select the data as of a particular number of seconds back in time
SELECT * FROM tasty_bytes.raw_pos.truck_dev
AT(OFFSET => -45);

SELECT $good_data_query_id;

--> select the data as of its state before a previous query was run
SELECT * FROM tasty_bytes.raw_pos.truck_dev
BEFORE(STATEMENT => $good_data_query_id);

```